

The C++ Programming Language

Historical Development, Design Principles, and Practical Application

Khadija Azzouzi

February 3, 2026

University of Siegen

Outline

Introduction

Historical Development of C++

Core Concepts and Design Principles

Applications and Use Cases

Tooling and Ecosystem

Implementation of the Search Tool

Performance Observations

Conclusion

Introduction

- C++ between low-level systems programming and high-level abstraction
- Often perceived as complex or outdated
- Still essential in performance- and resource-critical domains

Historical Development of C++

- Origin in the late 1970s as *C with Classes*
- Motivation: add abstraction without sacrificing performance
- Strong emphasis on compatibility with C
- Early design driven by systems programming needs

- ISO standardization beginning in the 1990s
- Continuous evolution through language standards
- Backward compatibility as a key design constraint
- Core philosophy remained largely unchanged

Core Concepts and Design Principles

Distinction from C

- C++ as a superset of C, but conceptually distinct
- Stronger static type system than C
- Support for user-defined types with value semantics
- Deterministic object lifetime
- Separation of interface and implementation through abstractions

- RAII as a central idiom in C++
- Resource lifetime bound to object lifetime
- Automatic cleanup via destructors
- Reduces error-prone manual resource handling

- Templates enable generic programming
- Compile-time type checking and instantiation
- No runtime overhead compared to handwritten code
- Important for reuse and performance
- Errors detected at compile time rather than runtime

Applications and Use Cases

- Systems software and operating systems
- Compilers and language tooling
- Embedded and performance-critical systems
- Chosen where efficiency and control are essential

Tooling and Ecosystem

- C++ development relies heavily on tooling
- Build systems and compilers are essential
- Tooling complexity reflects language flexibility
- Managing complexity is part of C++ development

Implementation of the Search Tool

Task and Design Goals

- Grep-like command-line search tool
- Recursive directory traversal
- Regular expression matching
- Context output and formatting options

- Use of `std::filesystem` for traversal
- Explicit exclusion of binary files
- Standard-library-only implementation
- Emphasis on clarity and portability

- File handling via `std::ifstream`
- Automatic resource cleanup via scope exit
- No manual memory or resource deallocation
- Demonstrates RAI in practical code

Performance Observations

- Performance dominated by I/O and filesystem access
- Regex matching adds noticeable overhead
- Reading full files simplifies context handling
- Trade-off between simplicity and memory usage

Conclusion

- C++ remains relevant in carefully chosen domains
- Strengths and complexity are closely connected
- Core design principles remained stable over decades
- Effective use requires deliberate design decisions

References

Stroustrup, B. (1984). The C++ Programming Language – Reference Manual.

Stroustrup, B. (1995). Why C++ is not just an object-oriented programming language.

Stroustrup, B. (1999). An overview of the C++ programming language.

Stroustrup, B. (2013). The C++ Programming Language (4th ed.).

Brunner, T., Porkoláb, Z. (2017). Programming language history: Experiences based on the evolution of C++.

Nienhuis, K., Memarian, K., Sewell, P. (2016). An operational semantics for C/C++11 concurrency.

Stroustrup, B. (2025). 21st Century C++.